

Project Documentation

Boston Image – Image Contrast Enhancement
MATLAB Image Processing Project

This document describes the purpose, methodology, environment requirements, execution steps, and results of the *image_processing.m* MATLAB script, which compares four contrast-enhancement techniques on a night-time photograph.

1. Project Purpose

The purpose of this project is to demonstrate and visually compare four classical image-contrast-enhancement techniques — Gamma Correction, `imadjust` (contrast stretching), `histeq` (global histogram equalization), and `adapthisteq` (adaptive/local histogram equalization) — applied to the Value (brightness) channel of an HSV-converted RGB photograph (*boston night.jpg*). By isolating adjustments to the Value channel and leaving Hue and Saturation unchanged, the project preserves original color while allowing a controlled comparison of how each algorithm redistributes brightness and local contrast in a low-light, high-dynamic-range night scene.

This kind of comparison is directly relevant to practical problems such as improving visibility in surveillance or night-photography footage, preprocessing images for computer-vision pipelines, and teaching the trade-offs between global and local, linear and non-linear enhancement strategies.

2. Methodology

2.1 Color-Space Conversion

The source image is read with `imread` and converted from RGB to HSV using `rgb2hsv`. The resulting HSV image is split into its three constituent channels: Hue (H), Saturation (S), and Value (V). Working in HSV rather than RGB allows brightness/contrast operations to be applied to the V channel alone, without altering hue or color saturation.

2.2 Contrast Enhancement Techniques

#	Technique	Function	Mechanism
1	Gamma Correction	$V.^{(1/2)}$	Non-linear power-law transform ($\gamma=0.5$) that brightens mid-tones
2	Contrast Stretching	<code>imadjust(V)</code>	Maps the input intensity range to the full output range (default [0,1]), linear
3	Global Histogram Equalization	<code>histeq(V)</code>	Redistributes pixel intensities so the output histogram is approximately uniform
4	Adaptive Histogram Equalization	<code>adapthisteq(V)</code>	Performs histogram equalization within small local tiles (CLAHE-style), improving

2.3 Recombination and Display

For each technique, the adjusted V channel is recombined with the original, unmodified H and S channels using `cat(3, H, S, V_adjusted)`. Each recombined HSV image is converted back to RGB with `hsv2rgb` and rescaled from the [0,1] double range to 8-bit integers with `uint8(255 * ...)` for standard image display. Finally, all four resulting images are displayed together using `montage({ ... })` with a descriptive title, enabling direct side-by-side visual comparison.

3. MATLAB Version & Required Toolboxes

Requirement	Details
MATLAB Version	R2018b or later (verified working on R2021a and newer; earlier releases with the listed toolbox support may work)
Required Toolbox	Image Processing Toolbox — provides <code>rgb2hsv</code> , <code>hsv2rgb</code> , <code>imadjust</code> , <code>histeq</code> , <code>adapthisteq</code> , and <code>imopen</code>
Other Toolboxes	None. Base MATLAB plus Image Processing Toolbox is sufficient.
Operating System	Platform-independent (Windows, macOS, Linux) — no OS-specific dependencies.

4. Project Files & Dependencies

File / Folder	Description
code/image_processing.m	Main MATLAB script implementing the full workflow (import, HSV conversion, four enhancement steps, and export)
data/boston night.jpg	Required input image (RGB, night photograph of Boston). Must be supplied by the user and placed in the <code>data</code> folder
data/README_DATASET.txt	Note describing the dataset dependency and what to do if the image file is not present.
BostonImageContrastEnhancement.mtlx	MATLAB Project archive bundling the script and data folder for easy import into MATLAB (Project Compiler required)

Note: the original source material provided for this documentation consisted of the MATLAB script and a README description only; the referenced GitHub repository containing the actual *boston night.jpg* file was not reachable at packaging time. The image must be supplied by the user before running the script — see Section 6 (Execution Steps).

5. Script Listing (image_processing.m)

```
% Import image and convert to HSV
img = imread("boston night.jpg");
imgHSV = rgb2hsv(img);

% Separate the HSV channels
H = imgHSV(:,:,1);
S = imgHSV(:,:,2);
V = imgHSV(:,:,3);

% 1. Gamma Correction
imgGamma_V = V.^(1/2);
imgGamma_HSV = cat(3, H, S, imgGamma_V);
imgGamma = uint8(255 * hsv2rgb(imgGamma_HSV));
```

```

% 2. imadjust

imgAdjust_V = imadjust(V);

imgAdjust_HSV = cat(3, H, S, imgAdjust_V);

imgAdjust = uint8(255 * hsv2rgb(imgAdjust_HSV));

% 3. histeq

imgEq_V = histeq(V);

imgEq_HSV = cat(3, H, S, imgEq_V);

imgEq = uint8(255 * hsv2rgb(imgEq_HSV));

% 4. adapthisteq

imgAdapt_V = adapthisteq(V);

imgAdapt_HSV = cat(3, H, S, imgAdapt_V);

imgAdapt = uint8(255 * hsv2rgb(imgAdapt_HSV));

% Display comparison montage

montage({imgGamma, imgAdjust, imgEq, imgAdapt});

title('Gamma, imadjust, histeq, and adapthisteq');

```

6. Execution Steps

- 1 Install MATLAB (R2018b or later) with the Image Processing Toolbox enabled.
- 2 Extract or open the MATLAB Project archive (BostonImageContrastEnhancement.mlpj) via MATLAB > Project > Open Project, or simply copy the code/ and data/ folders to a working directory.
- 3 Place a file named exactly "boston night.jpg" (RGB image) into the same folder as image_processing.m, or edit the imread() line to point to your image's path.
- 4 Set the MATLAB Current Folder to the project's code/ directory.
- 5 Open image_processing.m in the MATLAB Editor and click Run (or type "image_processing" at the Command Window prompt).
- 6 MATLAB will read the image, perform all four contrast-enhancement computations, and open a Figure window showing a 2x2 (or 1x4) montage of the four results with the title "Gamma, imadjust, histeq, and adapthisteq".
- 7 Optionally, inspect the individual output variables (imgGamma, imgAdjust, imgEq, imgAdapt) in the MATLAB Workspace, or export the montage figure via File > Save As for reporting purposes.

7. Results

Running the script produces four enhanced RGB images and a single comparison montage:

Output Variable	Technique	Expected Visual Characteristic
imgGamma	Gamma Correction (<code>&gamma=0.5</code>)	Preserves shadows and mid-tones smoothly; highlights remain largely unchanged
imgAdjust	<code>imadjust</code>	Increases global contrast by stretching the intensity range; can clip very dark/bright
imgEq	<code>histeq</code>	Strong global contrast boost; can over-amplify noise and produce an unnatural look
imgAdapt	<code>adapthisteq</code>	Improves local/regional contrast (e.g., building facades, lit windows) while limiting

Because the specific pixel content of *boston night.jpg* was not available at packaging time, exact numeric/histogram results cannot be reported here. The qualitative comparison above reflects the well-documented, expected behavior of each MATLAB function and should be confirmed by running the script against the actual image, then visually reviewing the resulting montage figure.

8. Conclusion

This project provides a compact, reproducible MATLAB workflow for comparing global parametric (gamma), global automatic (`imadjust`), global statistical (`histeq`), and local adaptive (`adapthisteq`) contrast-enhancement techniques on a real-world low-light photograph. The HSV-channel-isolation approach ensures a fair, color-preserving comparison, making the results directly interpretable for selecting an appropriate enhancement method in downstream image-processing or computer-vision applications.